Personal Server Dashboard

Documentation for Leandro's server dashboard

Last updated: November 1, 2026

By Leandro Chamorro

Contents

1 Objective **2**

2 Project scope **2**

3 General plan **2**

4 Current state **3**

4.1 Open the project 3

4.2 Take a sample 3

4.3 Save a sample 3

4.4 Check the saved data 3

5 Final product acceptance criteria **3**

6 Future work **4**

7 References **4**

1 Objective

This document describes the plan and implementation of a dashboard for monitoring the components of my personal server. The project is also an opportunity to practice using Python, Streamlit and SQLite.

The project grew out of the need to check the status of my server while I am away. The goal is to display its measurements in a simple interface that I can access easily from any other device.

2 Project scope

- Capture CPU, RAM and disk usage approximately once per second.
- Display each measurement as a percentage, together with the time of the latest sample.
- Store samples locally using SQLite.
- Refresh the dashboard every two seconds and show a warning when the latest sample is at least five seconds old.
- Make the project reproducible on another computer.

The current CPU measurement takes 0.5 seconds. This is the measurement window, not the interval between samples. The collector will account for this time when scheduling each one-second cycle.

3 General plan

The following plan is the complete planned work for the project divided into ten work blocks. Each block covers a specific part of the project and defines the result required before moving to the next one.

Work block	Description	Acceptance criteria
1 Prepare the environment	Create the project folder and its own Python virtual environment, install psutil.	The project's Python interpreter runs and imports psutil.
2 Measure the PC	Collect CPU, RAM and disk usage with the time of each measurement.	A sample contains the three usage percentages and its timestamp.
3 Store measurements	Set up a local database and save measurements.	Measurements remain available after the program closes.
4 Read stored measurements	Retrieve the latest measurement from the database.	The latest sample can be retrieved, and missing data are handled clearly.
5 Collect continuously	Run measurement collection and storage automatically at regular intervals.	Samples are saved continuously, and collection can be stopped cleanly.
6 Display live measurements	Build a Streamlit dashboard to display the latest measurements.	The dashboard shows current values and updates automatically.
7 Show data freshness	Indicate whether measurements are recent, outdated or unavailable.	The dashboard clearly distinguishes fresh, outdated and missing data.
8 Verify recovery and persistence	Check the monitor when collection stops and restarts.	Previous measurements are preserved, and live updates resume after a restart.
9 Automate the critical checks	Create automated tests for data storage and freshness.	The main storage and freshness checks pass independently of private measurements.
10 Reproduce the installation	Verify the documented setup in a new environment.	The monitor and its tests run successfully using the documented instructions.

4 Current state

Blocks 1,2 and 3 are complete, the project environment is ready, the code can measure CPU, RAM and disk usage, and samples can be saved in a local SQLite database. The next step is block 4 which has not yet been implemented.

At this stage, measurements are taken manually from the PC running the code. Each sample includes three usage percentages and a UTC timestamp. The following guide shows how to take a sample, save it and confirm that it is stored.

4.1 Open the project

Open PowerShell in the project's local-system-monitor folder. Run all commands below from that folder.

4.2 Take a sample

Run the measurement script:

```
.\.venv\Scripts\python.exe .\measure.py
```

The command displays CPU, RAM and disk usage with a UTC timestamp. It takes one sample without saving it.

4.3 Save a sample

First, prepare the database:

```
.\.venv\Scripts\python.exe .\storage.py
```

The message "Database ready" means that the database is ready. Existing samples are preserved. Then take and save a new sample:

```
.\.venv\Scripts\python.exe -c "from measure import take_sample; from storage import save_sample;
    save_sample(take_sample()); print('Sample saved')"
```

This command loads the measurement and storage functions, collects one sample and saves it. The message "Sample saved" confirms that the operation finished. Each run adds a new sample to data/monitor.sqlite.

4.4 Check the saved data

Count the samples in the database:

```
.\.venv\Scripts\python.exe -m sqlite3 .\data\monitor.sqlite "SELECT COUNT(*) FROM measurements;"
```

The result shows the number of saved samples. For example, `(1,)` means that one sample is stored. The count increases after each successful save. This checks that data are stored; retrieving the latest sample will be added in block 4.

5 Final product acceptance criteria

- The collector saves samples periodically and continues running when the browser is closed.
- The page displays percentages and the time of the latest sample, or a clear message when there are no samples.
- Installation and startup instructions work in a different environment.

6 Future work

After completing the local monitor, I would like to display processes left running on the server and sending email alerts when resource usage exceeds a defined threshold. I would also like to explore the idea of a button that turns off the device or kills all processes in case I detect a dangerous spike in the components usage, this idea may be descarted or just delegated to an alert to shut it down through a secure connection with the server to ensure security.

7 References

psutil documentation. (n.d.). Retrieved November 1, 2026, from `https://psutil.readthedocs.io/stable/`

Python Software Foundation. (n.d.-c). *datetime—Basic date and time types*. Python 3.13 documentation. `https://docs.python.org/3.13/library/datetime.html`

Python Software Foundation. (n.d.-f). *sqlite3—DB-API 2.0 interface for SQLite databases*. Python 3.13 documentation. `https://docs.python.org/3.13/library/sqlite3.html`

Python Software Foundation. (n.d.-g). *Virtual environments and packages*. Python 3.13 documentation. `https://docs.python.org/3.13/tutorial/venv.html`

Streamlit. (n.d.). *Working with fragments*. Retrieved November 1, 2026, from `https://docs.streamlit.io/develop/concepts/architecture/fragments`